

PhishGuard

How It Works, and How We Built It

A detailed, plain-English explanation — with a focus on the Artificial Intelligence

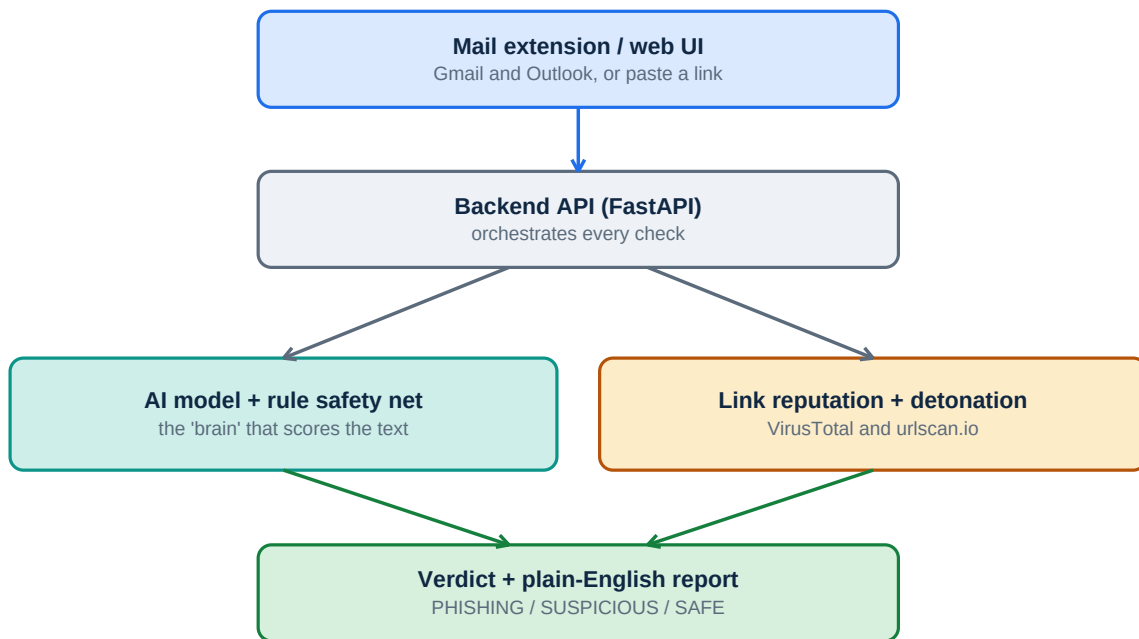


Figure 1 — The whole system. The AI model (green-blue box) is the brain that scores the email text.

Part A — The Role of AI in PhishGuard

1. What problem are we solving?

Phishing emails try to trick a person into clicking a link and handing over a password or downloading malware. They are hard to catch because attackers constantly change their wording, their fake domains, and their tricks. We wanted a tool that looks at an email and tells the user, in plain language, whether it is phishing and *why* — and that can safely show what a suspicious link would do.

2. Where exactly is AI used?

PhishGuard uses Artificial Intelligence in the form of **Machine Learning (ML)**. The AI's single job is to look at an email and estimate the **probability that it is phishing** (a number from 0% to 100%). It is the green-blue box in Figure 1 — the brain that scores the text. Everything else in the system (the link checks, the sandbox, the web page, the extension) supports or acts on that AI decision.

The specific AI technique is a **supervised classification model called Logistic Regression**. 'Supervised' means we teach it using examples that are already labelled phishing or legitimate. 'Classification' means its output is a category (phishing vs. legitimate). It is a classical, well-understood ML method — not a large language model and not deep learning.

3. Why use AI here instead of just writing rules?

We could try to write fixed if-then rules for every phishing trick, but there are too many, and attackers invent new ones. AI is better suited because it can:

- **Weigh many clues together.** An email is rarely phishing because of one thing; it is the *combination* of urgency, a fake domain, and a password request. The model learns how much each clue matters and adds them up.
- **Learn from examples.** Instead of us hand-tuning numbers, the model figures them out from data, and can be re-trained as new phishing appears.
- **Give a confidence score, not just yes/no.** A probability lets us say 'very likely phishing' (90%) vs. 'a bit suspicious' (40%).

4. Why this *kind* of AI (and not a fancier one)?

We deliberately chose simple, classical ML rather than deep learning or a large language model, because Logistic Regression is **explainable** (we can see exactly which clue influenced the decision and by how much), **fast**, runs on an ordinary laptop, and needs very little data. For a security tool, being able to explain *why* an email was flagged is as important as the flag itself.

Part B — How the AI Actually Works

The AI works in three steps: turn the email into numbers, learn from labelled examples, and then predict on new emails. Figure 2 shows the flow.



Figure 2 — The AI model pipeline

Step 1 — Turning an email into numbers (feature engineering)

A computer cannot 'read' an email the way we do. So the most important human step was deciding which measurable clues to give the AI. We chose **19 clues** (called *features*) that tend to differ between phishing and real email, and wrote code to measure each one. The email becomes a list of 19 numbers.

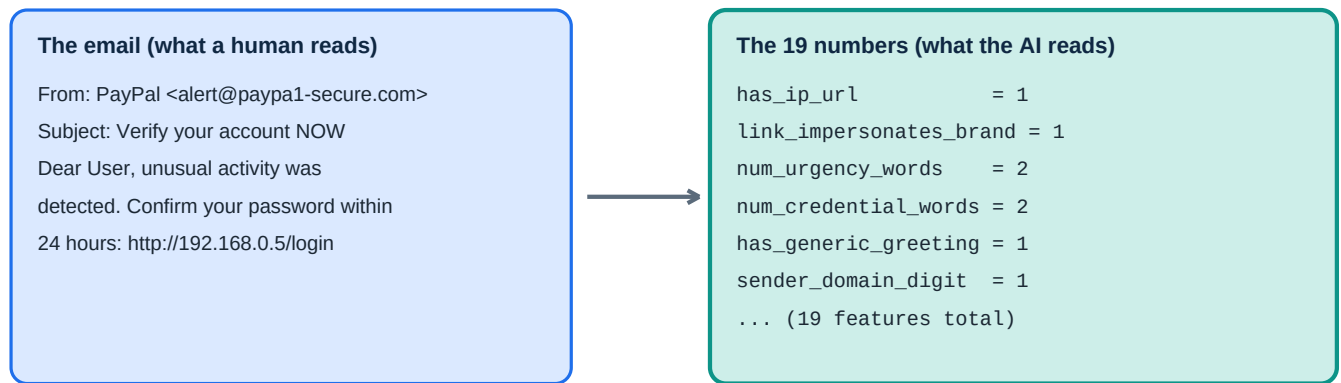
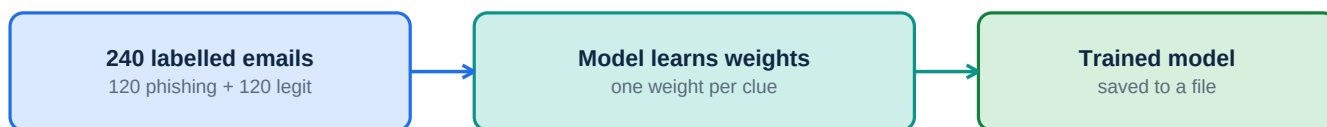


Figure 3 — The same email as a human sees it (left) and as the AI sees it (right)

The 19 clues fall into three groups: **link clues** (a raw IP-address link, a shortener, the '@' trick, a brand-impersonating domain, link length, ...), **language clues** (urgency words, threat words, password/login requests, a generic 'Dear User' greeting, ...), and **sender clues** (a free-email address, a display name that claims a brand the domain does not match, digits/hyphens in the domain, ...). Choosing good clues is where human expertise meets the AI.

Step 2 — Teaching the model (training)

To 'learn', the model needs examples. We gave it a dataset of **240 emails** — 120 phishing and 120 legitimate — each already labelled with the correct answer (1 = phishing, 0 = legitimate). The model looks at all of them and adjusts a **weight** for each of the 19 clues, so that the weighted combination best separates phishing from legitimate. That automatic adjustment *is* the 'learning'.



25% of the emails are hidden during training and used only to test the model honestly afterwards.

Figure 4 — How the model is trained

To make sure the model genuinely learned (rather than memorised), we **held out 25% of the emails** as a test set the model never saw during training, and also used **5-fold cross-validation** (training and testing five times on different splits). A weight that ends up large and positive means 'this clue strongly points to phishing'. In our model the strongest learned signals were a generic greeting, digits/hyphens in the sender domain, password/login requests, urgency and threat language, and brand mismatch — exactly the clues a human analyst would use, which is a good sign the model learned sensibly.

Step 3 — Making a decision on a new email (prediction)

For a brand-new email, the system measures the same 19 clues, multiplies each by its learned weight, adds them up, and passes the total through the **logistic function**, which squashes any number into a probability between 0 and 1. If the probability is high the email is labelled PHISHING; in the middle, SUSPICIOUS; low, LIKELY SAFE.

Step 4 — Explaining the decision (why this is special)

Because every clue has a weight, we can list *which* clues pushed a particular email's score up. That is exactly how PhishGuard produces its plain-English 'Why' reasons (for example, 'urgent, pressuring language' or 'a link impersonating a known brand'). Many AI systems are black boxes; choosing Logistic Regression let us make the AI's reasoning visible to the user.

5. AI combined with rules (a hybrid 'brain')

Pure AI has a weakness: it reflects the examples it was trained on, and can miss an obvious case that looks unlike its training data. So we added a small set of **hand-written rules** for high-confidence patterns (for example, a raw-IP link together with a password request, or a domain that clearly impersonates a brand). These rules can only *raise* suspicion, never lower it. The final decision is therefore AI + rules + live link intelligence working together — more trustworthy than any one alone. We discovered the need for this the hard way (see Part C).

Part C — How We Built the Whole System, Step by Step

We built PhishGuard in four phases, always keeping a working product, and also attempted a fifth, more advanced component.

Phase 1 — The AI model on its own

We first generated a labelled dataset, wrote the feature-extraction code (the 19 clues), trained the Logistic Regression model, evaluated it, and saved it to a file. At this point PhishGuard could already score an email and explain why — the heart of the project.

Phase 2 — A backend service and live link checks

We wrapped the model in a web service (FastAPI) so other programs could use it, and added live link-reputation checks from VirusTotal and urlscan.io. Now the system has two independent opinions: the AI judges the text, and threat-intelligence judges the links. A link proven malicious overrides the AI.

Phase 3 — Safe link detonation

We added the ability to open a suspicious link safely: an offline structural analysis that always runs, plus live 'detonation' through the urlscan.io sandbox, which opens the link away from the user's computer and returns a screenshot and verdict.

Phase 4 — A friendly web app and a Gmail/Outlook extension

We built a clean web page where the user pastes an email or link and sees a colour-coded result, and a Chrome extension that puts a 'Check email' button right inside Gmail and Outlook, showing the verdict without leaving the inbox.

Attempted — A self-hosted isolated sandbox on AWS

We also tried to run our own detonation sandbox on an isolated AWS cloud machine (EC2) inside a private network (VPC), built with Terraform. The cloud infrastructure deployed successfully and the backend connected to it, but the browser that opens links inside the isolated container never started (a missing-library error that we could not resolve in the available time). We therefore removed the cloud machine to avoid cost and used urlscan.io as the working detonation engine. It is included honestly as an attempted, unfinished component.

The three problems we found and fixed (the best learning)

Testing on realistic emails exposed three real weaknesses, each of which we diagnosed and repaired — and each taught us something about AI:

- **Short phishing slipped through.** The model had only seen long, wordy phishing, so it had quietly learned 'long = phishing' and rated a one-line phishing email as safe. We fixed it by adding varied (short) phishing to the training data and adding the rule safety net. Lesson: a model inherits the biases of its training data.
- **A fake link in the body was missed.** A clear phishing email scored only 26% because the fake PayPal domain was in a link in the body, while the model only checked the sender. We added a new clue and rule to detect brand impersonation in body links; the same email then scored 100%. Lesson: phishing signals can hide anywhere in a message.
- **The sandbox rejected scans.** Live detonation first failed because free urlscan accounts reject 'private' scans and because fake domains do not resolve. We switched to 'unlisted' scans and added a friendly message. Lesson: real services have constraints you only discover by testing.

Part D — What Each Part of the Code Does

File / folder	What it does
src/make_dataset.py	Generates the 240 labelled example emails to train on
src/features.py	Turns an email into the 19 numeric clues (the AI's input)
src/train.py	Trains the AI model, evaluates it, and saves it to a file
src/predict.py	Scores a new email and produces the verdict + 'Why' reasons
src/rules.py	The rule safety net that escalates obvious red flags
src/link_check.py	Asks VirusTotal and urlscan.io about each link
src/detonate.py	Offline link analysis + live urlscan detonation
src/api.py	The FastAPI backend that ties everything together
web/index.html	The web page the user interacts with
extension/	The Chrome extension for Gmail and Outlook
sandbox/aws/	The attempted isolated-cloud sandbox (Terraform, etc.)

To run it: install the requirements, train the model once, then start the backend and open the web page in a browser. With a free urlscan.io key, live detonation is enabled too.

Part E — What the AI Did, and Honestly Did Not, Do

What the AI did: it learned, from labelled examples, how to weigh 19 clues and output a phishing probability with an explanation — the core decision of the whole tool.

What the AI did not do: it does not 'understand' an email the way a person does; it does pattern-matching on the clues we engineered. It does not open links, browse the web, or write the report — those are done by other parts of the system. And because it was trained on a small, clean, synthetic dataset, its near-perfect test scores are optimistic; on messy real-world email the accuracy would be lower. The honest next step for a production-grade model is to retrain it on large, real phishing datasets.

Conclusion

In PhishGuard, AI is the decision-making brain: a transparent machine-learning model that turns an email into 19 measurable clues and predicts, with an explanation, how likely it is to be phishing. Around that brain we built a backend, live link intelligence, safe link detonation, a web app, and an in-inbox extension — and we honestly recorded the one advanced component we attempted but could not finish. Just as importantly, by testing on realistic emails we found and fixed real weaknesses in the AI, which is the clearest demonstration of understanding how and why it works.